Ders 17 Çalışma Özeti

Ders 17 Çalışma Özeti

Genel Konular

- Heap ağacının (heap tree) tekrarı: sola dayalı, dengeli ve heap özelliğine sahip ikili ağaç.
- Heap ağacına eleman ekleme ve `siftUp` fonksiyonu.
- Kökü silme (`removeMax`), `siftDown` fonksiyonu ve büyük çocuğu bulan yardımcı fonksiyon.
- Heap sort'un tekrarı, iki adım adım örnek ve diğer sıralama algoritmalarıyla karşılaştırma.
- Öncelikli kuyruk (priority queue): neden gerekli, hangi veri yapılarıyla yapılabilir ve neden binary heap tercih edilir.
- Huffman kodlaması (Huffman coding): önek kod (prefix code), kodlama, kod çözme ve sıkıştırma kazancı.
- Huffman ağacının öncelikli kuyruk mantığıyla adım adım kurulması.
- Ara sınavdaki algoritmik soruların çözüm mantığı: seyrek matris (sparse matrix), yazıcı kuyruğu, kuyruğu yığınla ters çevirme, karmaşıklık, sıralama algoritmasını tanıma ve üçlü min-heap'ten silme.

Hocanın Özellikle Vurguladığı Kısımlar

- Heap ağacında sol ve sağ çocuk arasında bir sıra kuralı yoktur. Bu, ikili arama ağacından (BST) farkıdır.
- Heap ağacı zaten dizide saklanmak için tasarlanmıştır. Sola dayalı ve dengeli olduğu için arada boş göz kalmaz.
- Sift up sırasında yalnızca yeni elemanın ebeveyn yoluna bakılır. Ağacın geri kalanı ilgilendirmez.
- Kökü silerken kökü adım adım aşağı taşımaya çalışmayın, bu zor bir yoldur. Kökü son elemanla değiştirin, sonra sift down yapın.
- Sift down iki aşamalıdır: önce iki çocuk birbiriyle karşılaştırılır, sonra seçilen çocuk ebeveynle karşılaştırılır.
- Sift down'da düğümün iki çocuğu mu, tek çocuğu mu var, yoksa yaprak mı, bunu eleman sayısı n'e bakarak anlarsınız.
- Heap sort, merge sort ile aynı zaman karmaşıklığındadır ama ek bellek kullanmaz.
- Öncelikli kuyruklarda en verimli yapı binary heap'tir. Ekleme de silme de $O(\log n)$'dir.
- Huffman kodlaması kayıpsızdır (lossless). Çok geçen sembol az bitle, az geçen sembol çok bitle gösterilir.
- Sınavlarda puan genelde çözümün mantığından ve tasarımından alınır. Noktalı virgül, `i++` unutmak gibi küçük hatalardan çok az puan gider. Ezberlemeyin, düşünmeye alışın, bol pratik yapın.
- Hazır kabul edilen fonksiyonlar (`enqueue`, `dequeue`, `isEmpty` gibi) sorulduğunda, bunların ne iş yaptığını bilmek yeterlidir. Nasıl yazıldıkları önemli değildir; `printf` kullanır gibi kullanın.

Kısa Tekrar Notları

- Dizide i. gözdeki düğümün çocukları $2i+1$ ve $2i+2$, ebeveyni $(i-1)/2$ (tam sayı bölmesi) gözündedir.
- **Ekleme:** yeni elemanı ilk boş göze (`Heap[n]`) koy, `siftUp` yap, `n`'i bir artır.

- **Sift up durma şartları:** eleman yerine yerleşti (ebeveyninden büyük değil) ya da köke geldi (`adr == 0`).
- **Kökü silme:** kökü sakla, son elemanı köke koy, `n`'i bir azalt, `siftDown` yap.
- **Sift down durma şartları:** eleman yaprağa geldi ya da çocuklarından büyük (max heap).
- **Çocuk sayısı testi:** `2i+2 < n` ise iki çocuk, değilse `2i+1 < n` ise tek çocuk, ikisi de değilse yaprak.
- Sift up ve sift down: en kötü $O(\log n)$. Heap sort: $O(n \log n)$, ek bellek $O(1)$.
- **Öncelikli kuyruk maliyetleri** (en kötü durum):

Yapı	Ekleme	En büyüğü/küçüğü çıkar	En büyüğü/küçüğü bul
Sıralı dizi	N	1	1
Sıralı liste	N	1	1
Sırasız dizi	1	N	N
Sırasız liste	1	N	N
Binary heap	$\log N$	$\log N$	1

- **Önek kod:** hiçbir kod başka bir kodun başı (öneki) olamaz. `0, 11, 101` önek koddur; `0, 1, 11` değildir.

Detaylı Açıklamalar

Heap Ağacına Eleman Ekleme (Sift Up)

- Slayttaki adımlar (ekrandan, 00:09:30):
  1. Yeni elemanı bir sonraki boş düğüme koy.
  2. Yeni eleman ebeveyninden küçük veya eşit olana kadar (ya da kök olana kadar) yukarı taşı (sift up).
- Örnek: max heap `[20, 16, 12, 5, 8]`, 35 ekleniyor.

```
35, 5. göze konur        : [20, 16, 12, 5, 8, 35]
ebeveyn (5-1)/2 = 2 -> 12 : 12 < 35, değiştir -> [20, 16, 35, 5, 8, 12]
ebeveyn (2-1)/2 = 0 -> 20 : 20 < 35, değiştir -> [35, 16, 20, 5, 8, 12]
adr = 0 (kök), dur
```

- En iyi durumda yeni eleman hiç yer değiştirmez. Örneğin 3 eklenirse ebeveyni 20'den küçük olduğu için yerinde kalır.
- En kötü durumda eleman köke kadar çıkar. Ağaç dengeli olduğu için yüksekliği $\log n$'dir. Ekleme $O(\log n)$'dir.
- `siftUp` kodu (ekrandan, 00:14:10). Hoca durma şartı için bir bayrak (flag) kullanmayı daha anlaşılır bulduğunu söyledi.

```c
void siftUp(int item, int Heap[], int adr)
{
    int stop = 0;
    while ((adr > 0) && (!stop))
    {
        parent = (adr - 1) / 2;
        if (Heap[parent] < item)
        {
            swap(&Heap[adr], &Heap[parent]);
            adr = parent;
        }
        else
            stop = 1;
    }
}
```

- Ekleme fonksiyonu yeni elemanı `Heap[n]`'e yazar, `siftUp`'ı çağırır ve eleman sayısını bir artırır.
- Ebeveyn formülü kontrolü: 8. gözdeki elemanın ebeveyni $(8-1)/2 = 3$; 7. gözdekinin ebeveyni $(7-1)/2 = 3$.

Kökü Silme (Sift Down)

- Örnek (ekrandan, 00:27:40): max heap `[28, 20, 12, 16, 8, 5]`, kök 28 siliniyor.

```
28 saklanır, son eleman 5 köke gelir, n = 5 : [5, 20, 12, 16, 8]
5'in çocukları 20 ve 12; büyüğü 20; 20 > 5   : [20, 5, 12, 16, 8]
5'in çocukları 16 ve 8; büyüğü 16; 16 > 5    : [20, 16, 12, 5, 8]
5 yaprak oldu, dur
```

- `removeMax` kodu (ekrandan, 00:27:40):

```c
int removeMax(int Heap[], int *n)
{
    int retValue = Heap[0];
    Heap[0] = Heap[*n - 1];
    (*n)--;
    siftDown(Heap, *n);
    return retValue;
}
```

- Silinen değer döndürülür. İsterseniz kullanırsınız (heap sort'ta olduğu gibi), istemezseniz dizi boyutu azaldığı için o göz artık yok sayılır.
- Hoca, sol çocuğun hep büyük olmasının tesadüf olduğunu göstermek için ikinci bir örnek yaptı. Orada büyük çocuk sağdaydı ve yer değiştirme sağ tarafta oldu.

Büyük çocuğu bulan fonksiyon (tahtadan, 00:59:20). Fonksiyonun adı slaytta `findSmallestChild`'dır ama max heap için büyük çocuğun indisini döndürür.

- Durum 1, iki çocuk var (`child2 < n`): `Heap[child2] > Heap[child1]` ise `child2`, değilse `child1` döndürülür.
- Durum 2, tek çocuk var (`child1 < n`): `child1` döndürülür.
- Durum 3, düğüm yaprak: `-1` döndürülür. Karşılaştırılacak kimse yok demektir.
- Burada `child1 = 2*i + 1`, `child2 = 2*i + 2`'dir. Dönen değer bir indistir (hoca buna "adres" de diyor).
- İki çocuk testi için `child2 < n` gerekir. Çünkü n elemanlı dizide son eleman $n-1$. gözdedir.
- Örnek: $n = 4$, 5 sayısı 1. gözde. $2 \cdot 1 + 2 = 4$, $4 < 4$ değil, iki çocuk yok. $2 \cdot 1 + 1 = 3 < 4$, tek çocuk var.
- Örnek: $n = 6$, 3. gözdeki düğüm. $8 < 6$ değil, $7 < 6$ değil. Düğüm yapraktır.

`siftDown` kodu (ekrandan, 01:06:30):

```c
void siftDown(int Heap[], int n)
{
    int parent = 0, child;
    child = findSmallestChild(Heap, parent, n);   /* büyük çocuğun indisi */
    while ((child != -1) && (Heap[child] > Heap[parent]))
    {
        swap(&Heap[child], &Heap[parent]);
        parent = child;
        child = findSmallestChild(Heap, parent, n);
    }
}
```

- Döngü iki durumda biter: `child == -1` (yapraktayız) ya da çocuk ebeveynden büyük değil.
- En kötü durumda eleman yaprağa kadar iner. Karmaşıklık $O(\log n)$'dir.

Heap Sort Tekrarı

- Max heap'te en büyük eleman her zaman köktedir. Kökü dizinin sonuna taşıyınca o eleman doğru yerine yerleşir.
- Döngü: `for (i = n-1; i > 0; i--) { swap(&Heap[0], &Heap[i]); siftDown(Heap, i); }`. i azaldıkça bakılan bölge küçülür.

Örnek 1 (tahtadan, 01:13:40). Max heap `[20, 10, 13, 8, 7, 11, 12]`.

Adım	Değişim	Sift down sonrası heap kısmı	Yerleşmiş kısım
1	20 ile 12	`13, 10, 12, 8, 7, 11`	`20`
2	13 ile 11	`12, 10, 11, 8, 7`	`13, 20`

- 1. adımda 12 köke gelir. Çocukları 10 ve 13'ten büyüğü 13'tür, yer değiştirirler. Sonra 12'nin tek çocuğu 11'dir, 12 büyük olduğu için durur.

Örnek 2. Max heap `[20, 10, 8, 7]`, $n = 4$.

i	Değişim	Sift down sonrası heap kısmı	Yerleşmiş kısım
3	20 ile 7	`10, 7, 8`	`20`
2	10 ile 8	`8, 7`	`10, 20`
1	8 ile 7	`7`	`8, 10, 20`

- Sonuç: `7, 8, 10, 20`.
- 1. satırda 7'nin çocukları 10 ve 8'dir, 10 ile yer değiştirir. Yeni yerinde ($i = 1$) $2 \cdot 1 + 1 = 3$ artık bölgede değildir ($n = 3$), yani 7 yapraktır.

Neden $O(n \log n)$? Sift down $O(\log n)$'dir ve n kez yapılır. Bu, heap özelliğine dayanır: max heap'te kök her zaman en büyüktür ve dengeli ağacın yüksekliği $\log n$'dir.

Sıralama Algoritmalarının Karşılaştırılması

Slayt tablosu (ekrandan, 01:25:30):

Algoritma	En iyi	Ortalama	En kötü	Ek bellek
Selection sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$
Insertion sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Shell sort	$O(n \log n)$	$O(n^{1.5})$	$O(n^{1.5})$	$O(1)$
Bubble sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$
Quicksort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(1)$
Mergesort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$
Heapsort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$

- Heap sort en kötü durumda merge sort kadar iyidir ve ek bellek istemez.
- Neredeyse sıralı ya da küçük diziler için insertion sort hâlâ en iyisidir. Heap sort neredeyse sıralı bir diziyi önce karıştırır.
- Büyük ve sırasız dizilerde ortalamada genelde en hızlısı quicksort'tur. Quicksort'un en kötü durumu ($O(n^2)$) dizi zaten sıralıyken ortaya çıkar. Quicksort ve merge sort sonraki derslerde anlatılacak.
- Karmaşıklıkta logaritmanın tabanı yazılmazsa 2 kabul edilir.

Öncelikli Kuyruk (Priority Queue)

- Normal kuyrukta sıradaki ilk işe hizmet edilir. Ama bazen sıradan önce gelmesi gereken işler vardır:
  - Acil serviste ağır yaralı bir hasta sıranın önüne alınır.
  - İşletim sisteminde çok uzun bir iş, arkasındaki küçük işleri uzun süre bekletmesin diye geri bırakılabilir.
  - Hastanede 80 yaş üstü hastalar sıranın önüne alınır.
- Üç yol:
  - **Sırasız yapı:** Gelen eleman sona eklenir ($O(1)$). Hizmet için her seferinde en öncelikli eleman aranır ($O(n)$).
  - **Sıralı yapı:** Gelen eleman insertion sort mantığıyla yerine konur ($O(n)$). Hizmet her zaman baştaki elemana verilir ($O(1)$). Örnek: `[2, 3, 4, 9]` dizisine 7 gelince 9 bir kaydırılır, 7 araya girer.

  - **Binary heap:** Ekleme sift up ile, en öncelikliyi çıkarma kök silme + sift down ile yapılır. İkisi de $O(\log n)$, en öncelikliyi bulmak $O(1)$'dir.
- Öncelikli kuyrukta en küçük mü en büyük mü öncelikli, ihtiyaca göre seçilir: min heap ya da max heap.
- Min heap tabanlı kuyruktan çıkarma adımları (slayt, ekrandan 02:07:30):
  1. Kökteki değeri döndür.
  2. Son elemanı köke taşı.
  3. Yeni kökü, çocuklarından küçük veya eşit olana kadar (ya da yaprak olana kadar) aşağı taşı.
- Heap sort'tan farkı: burada çıkan eleman saklanmaz, eleman sayısı her seferinde bir azalır.
- Örnek (tahtadan, 02:42:45): min heap `[3, 5, 9, 7, 6]`.

```
3'e hizmet et: 6 köke gelir         : [6, 5, 9, 7]
6'nın çocukları 5 ve 9; küçüğü 5    : [5, 6, 9, 7]   (6'nın çocuğu 7; 6 < 7, dur)
5'e hizmet et: 7 köke gelir         : [7, 6, 9] -> 6 ile değiştir -> [6, 7, 9]
6'ya hizmet et: 9 köke gelir        : [9, 7] -> 7 ile değiştir -> [7, 9]
```

- Hareket az ise (örneğin gelen elemanlar hep başa yerleşecekse) sıralı liste de iş görür. Ama genel durumda en verimlisi binary heap'tir.

Huffman Kodlaması

- 1952'de David Huffman tarafından geliştirildi.
- ASCII'de her karakter 8 bit tutar. Ama bir metinde bazı harfler çok (örneğin `a`), bazıları çok az (örneğin `ğ`) geçer. Az geçen harf için de 8 bit harcanır.
- Fikir: çok geçen sembolleri az bitle, az geçenleri çok bitle kodla. Toplamda yer kazanılır.
- Kodlar önek kod olmalıdır. Yoksa yan yana duran bitler çözülemez. Örneğin `a = 0` ve `b = 00` olsaydı `000` dizisinin ne olduğu anlaşılmazdı.
- JPEG ve MPEG de bir aşamasında Huffman kodlaması kullanır.

Örnek 1 (tahtadan, 02:23:30). Metin: boşluk 2, `a` 4, `b` 5, `c` 1 kez geçiyor; toplam 12 karakter.

```
Frekanslar küçükten büyüğe: c:1, boşluk:2, a:4, b:5
c + boşluk  -> 3
3 + a       -> 7
7 + b       -> 12 (kök)

Ağaç (sol 0, sağ 1):
12
+--0-- b
+--1-- 7
       +--0-- 3
       |      +--0-- c
       |      +--1-- boşluk
       +--1-- a

Kodlar: b = 0, a = 11, c = 100, boşluk = 101
```

- Normal hali: $12 \times 8 = 96$ bit.
- Huffman ile: $5 \cdot 1 + 4 \cdot 2 + 1 \cdot 3 + 2 \cdot 3 = 22$ bit.
- Kodlama örneği: `b`, `a`, `c`, boşluk, `a` dizisi `0 11 100 101 11` olur.
- Kod çözme ağaç üzerinde yapılır: kökten başlanır, `0` gelince sola, `1` gelince sağa gidilir. Yaprağa gelince o sembol yazılır ve köke dönülür. Önek kod olduğu için hiçbir belirsizlik çıkmaz.
- Gerçekte ağacın kendisi de saklanmalıdır. Ama büyük bir dosyada (örneğin 1 MB'lık bir resim) ağacın boyutu küçük kalır.
- Huffman kodlaması kayıpsızdır. JPEG'in kayıplı olması Huffman'dan değil, gözün fark etmeyeceği renk ve frekans azaltmasından gelir.

Huffman ağacının öncelikli kuyruk mantığıyla kurulması (tahtadan, 02:41:00). Frekanslar: `a`:3, `b`:3, `c`:1, `d`:1, `e`:2.

1. Harfler frekanslarına göre küçükten büyüğe bağlı listeye (linked list) yerleştirilir: `c:1, d:1, e:2, a:3, b:3`.
2. En küçük iki düğüm alınır. Biri sol, biri sağ çocuk olur, toplamları yeni köke yazılır. Yeni ağaç sırayı bozmayacak şekilde listeye geri eklenir.
3. Tek düğüm kalana kadar 2. adım tekrarlanır.

```
c:1 + d:1 -> 2       liste: e:2, (cd):2, a:3, b:3
e:2 + (cd):2 -> 4    liste: a:3, b:3, (e cd):4
a:3 + b:3 -> 6       liste: (e cd):4, (ab):6
4 + 6 -> 10          tek ağaç kaldı (kök 10)

Kodlar (sol 0, sağ 1): e = 00, c = 010, d = 011, a = 10, b = 11
```

- Eşit frekanslı düğümün başa ya da araya eklenmesi fark etmez. Hoca, araya eklemeyi göstermek için `(cd):2` düğümünü e'den sonraya koydu.
- En az geçen `c` ve `d` 3 bitle, en çok geçen `a` ve `b` 2 bitle gösterildi.
- Huffman ağacı sola dayalı değildir. Bu yüzden heap ağacı sayılmaz. Öncelikli kuyruk mantığı kullanılır ama yapı normal bir ikili ağaçtır.

Sınav Sorularının Çözüm Mantığı

- **Seyrek matris (sparse matrix):** Elemanlarının çoğu sıfır olan matristir. Bu, "yarısından fazlası sıfır" demek değildir; pratikte sıfır olmayanlar çok azdır.
  - Gerçek örnek: film öneri sistemleri. 10 milyon izleyici, 10 bin film; her kişi en fazla birkaç yüz film izler.
  - Yalnız sıfırdan farklı elemanlar (satır, sütun, değer) bağlı listede saklanır. Satır ve sütun saklanmalıdır, yoksa değerin matrisin neresinde olduğu bilinemez.
  - Soru: liste verildiğinde tamamen sıfır olan sütunları bul. Çözüm: sütun sayısı kadar bir dizi tut. Listedeki her elemanın sütun gözünü 1 yap (ya da artır). Sonunda sıfır kalan gözler tamamen sıfır olan sütunlardır.
- **Yazıcı kuyruğu:** Kuyrukta (iş numarası, sayfa sayısı) çiftleri var.
  - Sıradaki işe hizmet: baştaki işin sayfa sayısı kadar dönen bir döngü yap, sonra kuyruğun başını bir sonraki işe ilerlet.
  - İş silme: kuyrukta istenen iş numarasını ara. Bulursan önceki düğümü sonrakine bağla ve düğümü `free` et; bulamazsan uyarı ver. Ortadan silme gerektiği için bağlı liste daha kolaydır.
- **Kuyruğu yığınla ters çevirme:** Kuyrukta 1, 2, 3, 4 var. Sınavdaki kodda `dequeue` yerine yanlışlıkla `enqueue` yazıldığı için bu soru herkese doğru sayıldı. Doğru kod (tahtadan, 03:07:10):

```
while (!isQueueEmpty(queue))
{
    value = dequeueQueue(queue);
    pushStack(value, stack);
}
while (!isStackEmpty(stack))
{
    value = popStack(stack);
    enqueueQueue(value, queue);
}
```

- Sonuç: kuyruk `4, 3, 2, 1` olur.
- **Karmaşıklık sorusu** (ekrandan, 03:12:50):

```
int fonksiyon(int n) {
    int i, j, k = 0;
    for (i = n/2; i <= n; i++) {
        for (j = 2; j <= n; j *= 2) {
            k += n/2;
        }
    }
    return k;
```

```
}
```

- İç döngü j'yi ikiye katlar: $O(\log n)$. Dış döngü $n/2$'den n'e birer birer gider: $O(n)$. Cevap: $O(n \log n)$.
- **Sıralama algoritmasını tanıma:** Bir dizinin selection, insertion ve heap sort ile kısmen sıralanmış hâlleri veriliyordu.
  - Selection sort her turda en küçüğü bulup başa koyar. İlk gözde en küçük eleman olmayan sütun selection sort olamaz.
  - Insertion sort'ta baştaki bölüm kendi içinde sıralıdır ama henüz sıra gelmeyen elemanlar o bölüme girmemiştir.
  - Önce en kolay tanınanı (selection) belirleyip şıkları elemek işi kolaylaştırır.
- **Üçlü min-heap'ten (ternary min-heap) en küçüğü silme** (ekrandan, 03:16:55). Her düğümün en fazla 3 çocuğu vardır; dizi `[3, 13, 4, 91, 49, 29, 98, 32, 30, 15]`.
  - Son eleman 15 köke gelir. Kökün çocukları 13, 4, 91; en küçüğü 4; 15 ile 4 yer değiştirir.
  - 15'in yeni çocukları 32 ve 30; en küçüğü 30; $15 < 30$ olduğu için durur.
  - Cevap: `4 13 15 91 49 29 98 32 30`.
- **Uygun veri yapısı:**
  - Derleyicinin fonksiyonlardan geri dönmesi: yığın (stack).
  - Klavyede basılan tuşları ekrana yazma: kuyruk. İlk basılan tuş ilk yazılmalıdır; tuşlar bir tamponda (buffer) bekler.
  - Dengeli olmayan bir ikili ağacı saklama: bağlı liste. Dizi boş gözlerle dolardı.
  - Satrançta bir sonraki hamleyi belirleme: ağaç (karar ağacı, decision tree).

Ders sonrası notlar

- Hoca Huffman kurulumunda kökteki toplamın çocuklardan büyük olmasına bakıp yapıya "aslında öncelikli kuyruk" dedi. Doğrusu: öncelikli kuyruk, birleştirilecek alt ağaçları tutan listedir ve her adımda en küçük iki ağırlık çıkarılır (min öncelikli kuyruk). Huffman ağacının kendisi bir heap değildir; ders sonunda bu düzeltildi.
- Slayttaki min heap çıkarma adımında "çocuklarından büyük veya eşit olana kadar" yazıyor. Min heap için şart tersidir: yeni kök, çocuklarından küçük veya eşit olana kadar aşağı taşınır.
- Tabloda bubble sort'un en iyi durumu $O(n^2)$ yazıyor. Bir turda hiç yer değişmezse duran (bayraklı) bubble sort, sıralı dizide $O(n)$'de biter.
- Tabloda quicksort'un ek belleği $O(1)$ yazıyor. Özyineleme yığını yüzünden ortalamada $O(\log n)$, en kötü durumda $O(n)$ ek bellek gerekir.
- Logaritmanın tabanı Big-O sonucunu değiştirmez. Farklı tabanlar arasında yalnız sabit bir çarpan vardır.
- Hoca Huffman örneğinde kodlama dizisini okurken bir yerde "B'nin kodu 0, 0" dedi. Tahtadaki ağaca göre `b`'nin kodu tek bit `0`'dır.

Not: İsterseniz bu kayıtları NotebookLM gibi bir araca atıp daha istifadeli hale getirebilirsiniz.